

3SE2 – Übung 5

Themen: API-Dokumentation, Svelte Frontend für ein Spring Boot Projekt, Repetition Svelte

Du darfst diese Übung weiterhin alternativ in deinem Einzelprojekt umsetzen und dies anstelle der Übung abgeben.

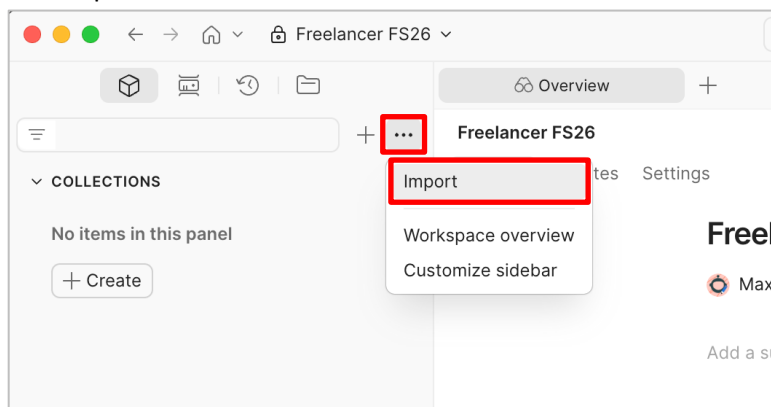
Folgendes wird erwartet:

- Ein Postman Collection mit sämtlichen Endpoints inklusive Beispieldaten.
- Veröffentlichte Postman Dokumentation (<https://documenter.getpostman.com/view/...>)
- Svelte 5 Frontend für Anzeige/Bearbeitung von in einer MongoDB gespeicherter Daten. Mindestumfang:
 - Page, die alle Dokumente (z. B. Companies) einer Collection anzeigt.
 - Page, auf der einer Collection neue Dokumente hinzugefügt werden können (z. B. neue Company).
 - Empfohlenes Vorgehen: Frontend-Beispiel auf Moodle herunterladen, gemäss Aufgabenstellung in das bestehende Projekt integrieren und anschliessend für das eigene Projekt anpassen.

In dieser Übung erstellen wir ein Frontend für das Projekt Freelancer4u. Dazu integrieren wir eine bestehende minimale Svelte App in das Freelancer-Projekt und erweitern dieses anschliessend.

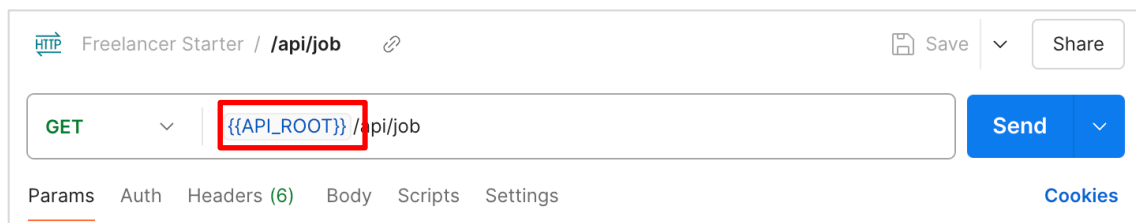
- Als Ausgangslage dient der letzte Stand des Freelancer4u-Projekts. Starte entweder mit deiner eigenen Lösung oder mit der Musterlösung der letzten Übung (siehe Moodle).
- Um die Endpoints übersichtlich zu dokumentieren, nutzen wir Postman. Dies hat bei der Entwicklung des Frontends den Vorteil, dass wir nicht im Java-Code nachschauen müssen, wie die Endpoints funktionieren. Ausserdem können wir diese unkompliziert testen.

Lade die Datei *Freelancer4U Starter.postman_collection.json* von Moodle und importiere diese in Postman via «Import»:

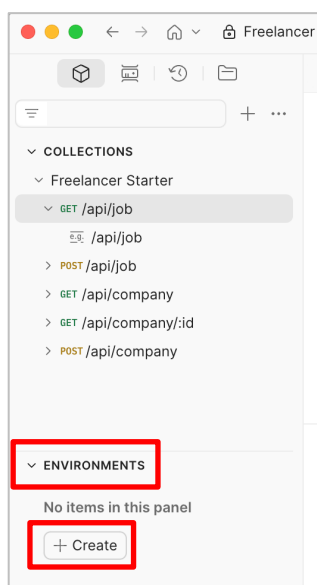


Folge dann den Anweisungen in Postman, um die Collection zu importieren. Wenn alles funktioniert hast, solltest du sie unter "Collections" sehen.

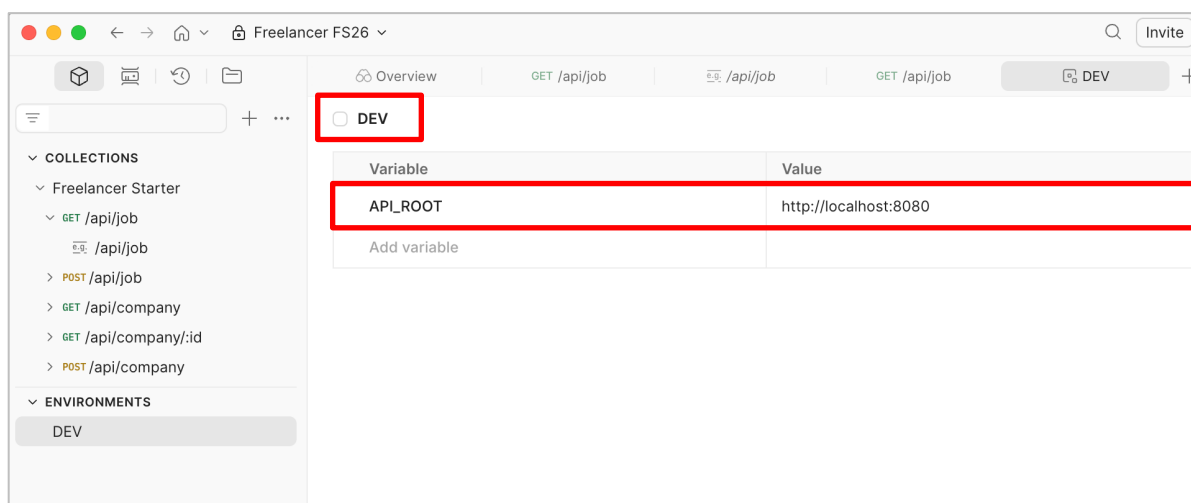
Für die Server URL wird in dieser Collection eine Variable verwendet, erkennbar an den doppelten geschweiften Klammern:



Diese globale Variable musst du manuell im Tab «Environments» konfigurieren. Erstelle zuerst ein neues Environment:



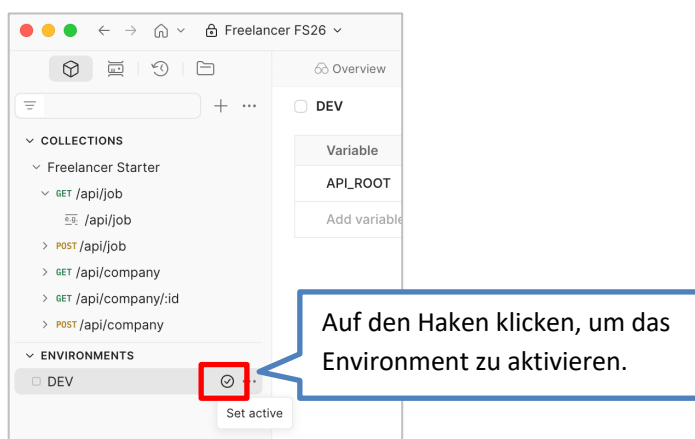
Nenne das Environment «DEV» und erstelle eine neue Variable:



Variablenname: API_ROOT

Value: http://localhost:8080

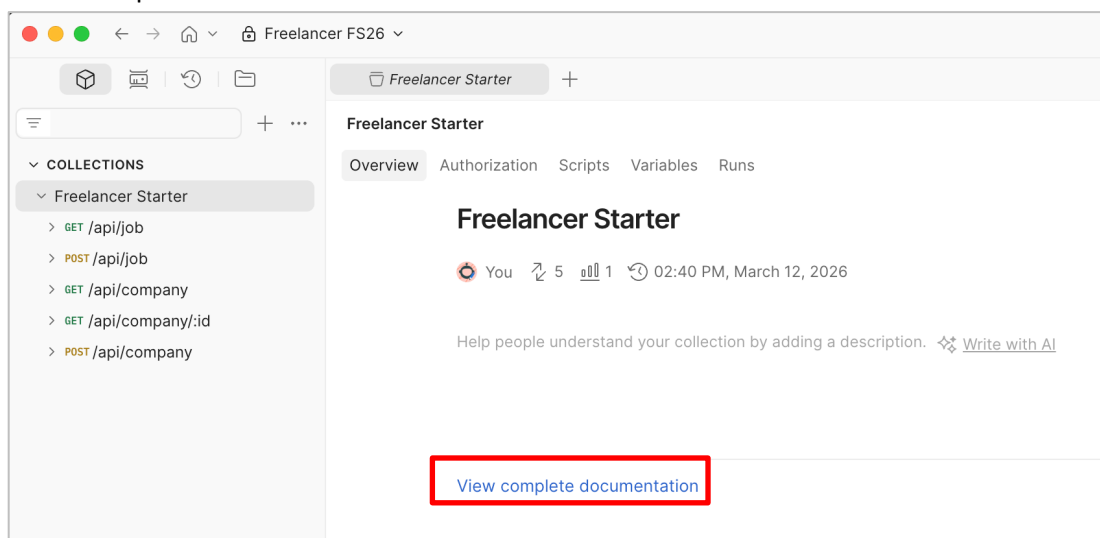
Zum Schluss musst du das Environment aktivieren:



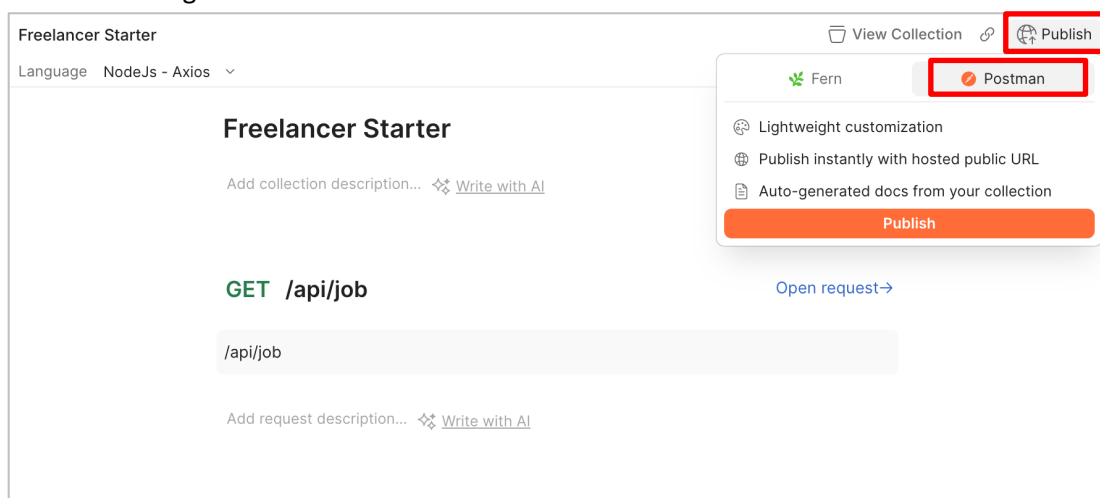
Teste, ob alles richtig konfiguriert ist, indem du das Backend startest und mit Postman sämtliche Endpoints aufrufst.

- c) Vervollständige die Collection und API-Dokumentation in Postman, indem du die bestehende Collection *Freelancer Starter* ergänzt. Orientiere dich dabei an den bereits vorhandenen Requests und am in der Vorlesung gezeigten Vorgehen.
- Füge der Collection die fehlenden Requests hinzu
 - PUT `/api/service/assignjob`
 - PUT `/api/service/completejob`
 - GET `/api/service/jobdashboard`
 - Erstelle zusätzlich Requests zum Testen der Requestparameter des Job-Endpoints:
 - GET `/api/job?min=...`
 - GET `/api/job?type=...`
 - Teste die erstellten Endpoints.
 - Füge jedem Request mindestens ein Beispiel hinzu.
 - Ergänze die Dokumentation: Füge jedem Request eine kurze Beschreibung hinzu.

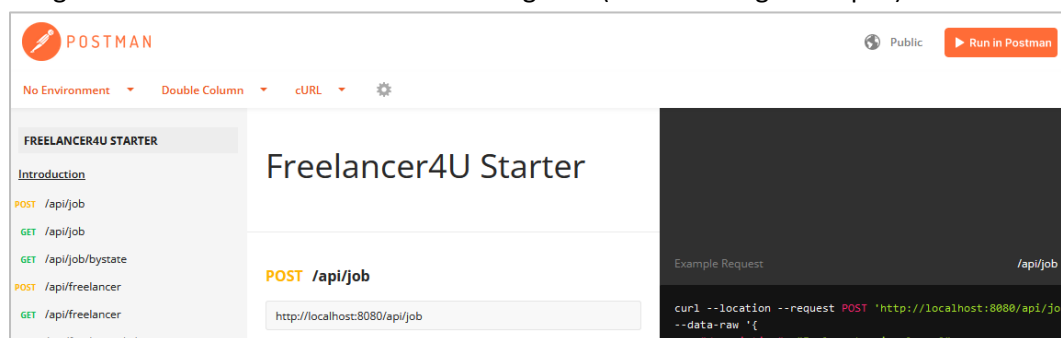
Veröffentliche deine API-Dokumentation. Wähle dazu die Collection in der Übersicht aus und klicke dann auf "View complete documentation":



Klicke auf "Publish", wähle «Postman» und folge den anschließenden Anweisungen. Verwende die Standardsettings.

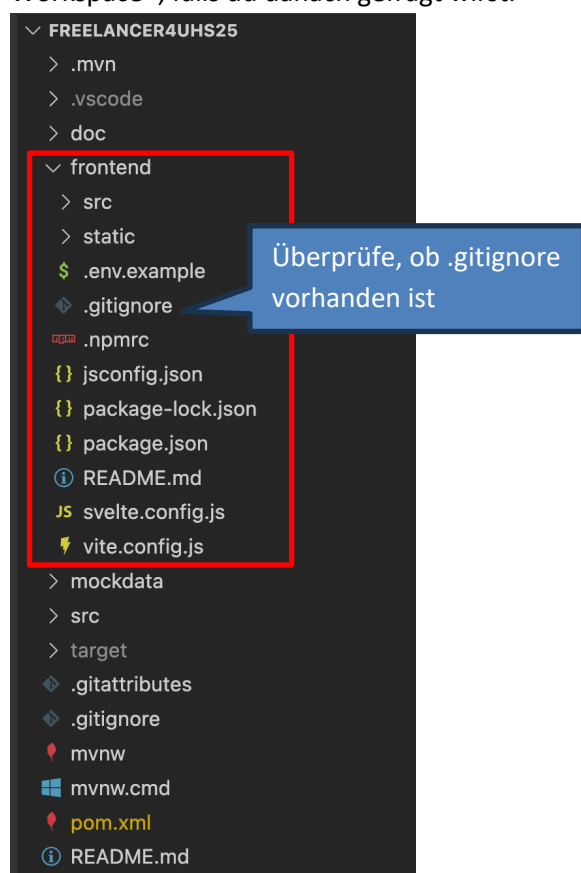


Am Ende erhältst du einen Link zur veröffentlichten Dokumentation, den du überprüfen solltest.
Die generierte Dokumentation sieht wie folgt aus (unvollständiges Beispiel):



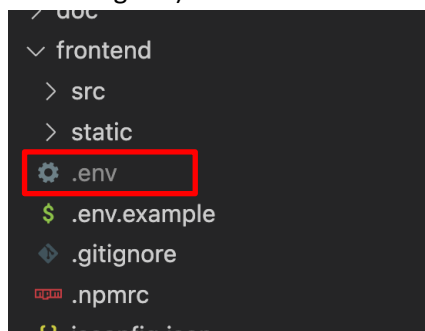
- d) Als nächstes musst du das bestehende Svelte Frontend in Betrieb nehmen. Lade das Projekt *freelancer4u-frontend* von Moodle, entzippe es auf deinem Rechner und verschiebe den Ordner *frontend* ins oberste Verzeichnis des Spring Boot Projekts, also den Ordner, der die Datei *pom.xml* enthält.

Wichtig: Verschiebe den gesamten Ordner und nicht nur den Inhalt, weil sonst unsichtbare Dateien wie beispielsweise *.gitignore* übersprungen werden. Wähle ausserdem «**Copy Folder**» und nicht «Add Folder to Workspace», falls du danach gefragt wirst.



Hinweis: Das Svelte Frontend ist eine eigenständige Anwendung, die unabhängig vom Spring Boot Backend läuft. Der Einfachheit halber verwalten wir beide Projekte im gleichen Git-Repository; man könnte auch zwei separate Repositories nutzen.

- e) Ähnlich wie beim Java-Backend verwenden wir eine separate Datei, um Variablen wie beispielsweise die URL des Backends zu verwalten:
- Erzeuge im Ordner *frontend* eine Datei mit dem Namen *.env* (beachte, dass der Dateiname mit einem Punkt beginnt):



- Kopiere den Inhalt von *.env.example* in die Datei *.env* :

```
# Backend API URL
API_BASE_URL=http://localhost:8080
```

Erklärung: Auf Variablen, die in *.env* definiert sind, kann im Svelte-Code wie folgt zugegriffen werden (siehe z.B. *frontend/src/routes/jobs/+page.server.js*):

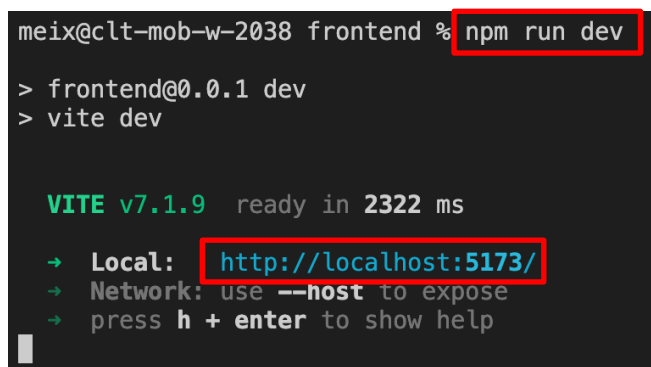
```
import 'dotenv/config';
const API_BASE_URL = process.env.API_BASE_URL;
```

- f) Um das Frontend zu starten, musst du einmalig alle Abhängigkeiten installieren. Öffne dazu ein Terminal (im Verzeichnis, in dem sich die Datei *package.json* befindet) und führe folgenden Befehl aus:

```
npm install
```

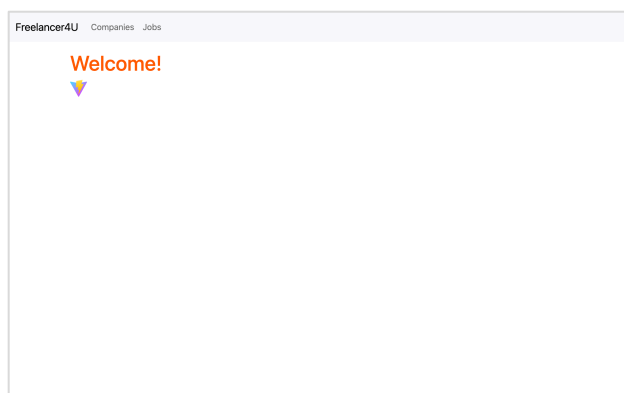
Anschliessend kannst du das Frontend mit folgendem Befehl starten:

```
npm run dev
```



- g) Starte nun die Spring Boot Anwendung (in einem neuen Terminal oder via die Extension «Spring Boot Dashboard») und öffne das Frontend in einem Browser.

Hinweis: Das Frontend läuft standardmässig auf dem Port 5173 (die URL ist also <http://localhost:5173>), das Backend auf dem Port 8080.



Gehe zur Seite "Jobs". Diese sollte wie folgt angezeigt werden:

The screenshot shows the 'Jobs' page in the Freelancer4U application. It features a 'Create Job' form with fields for Title, Description, Type, Earnings, and Company, and a 'Submit' button. Below the form is a table titled 'All Jobs' with columns for Title, Description, Type, Earnings, State, and CompanyId. The table contains three rows of data.

Title	Description	Type	Earnings	State	CompanyId
Main class	Implement Main Class	IMPLEMENT	145	NEW	18
Test Endpoints	Test all Endpoints	TEST	132	NEW	2
New job	Create a job	IMPLEMENT	150	NEW	23

Wenn in der Tabelle die in deiner Datenbank vorhandenen Jobs aufgelistet werden, hast du das Frontend richtig konfiguriert.

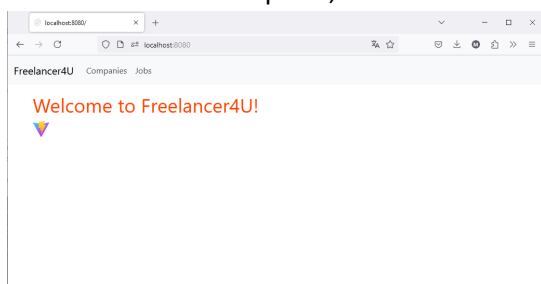
Teste auch noch, ob "Create Job" wie gewünscht funktioniert. Gib dazu Daten in das Formular ein und klicke auf *Submit*. Du solltest folgende Meldung sehen:

The screenshot shows the 'Jobs' page after a successful job creation. A green success message 'Job created successfully!' is displayed at the top. The 'Create Job' form is visible below the message. The 'All Jobs' table now includes an additional row for the newly created job, 'Test POST /api/job', with a description 'Test create job endpoint' and earnings of 155.

Title	Description	Type	Earnings	State	CompanyId
Main class	Implement Main Class	IMPLEMENT	145	NEW	18
Test Endpoints	Test all Endpoints	TEST	132	NEW	2
Test POST /api/job	Test create job endpoint	TEST	155	NEW	18
New job	Create a job	IMPLEMENT	150	NEW	23

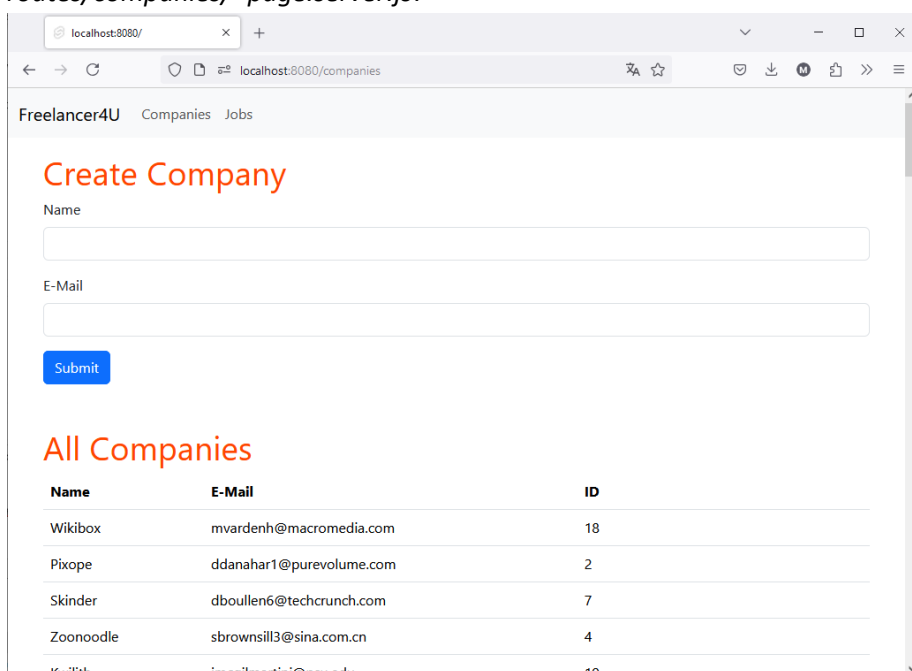
Überprüfe, ob der neu erstellte Job danach in der Liste angezeigt und in deiner Datenbank vorhanden ist.

- h) Öffne die Datei `routes/+page.svelte` und ändere den Titel von «Welcome!» zu «Welcome to Freelancer4U!». Überprüfe, ob der neue Titel angezeigt wird:



- i) Der Anfang ist geschafft! Du hast den kompletten Technologiestack in Betrieb genommen und kannst nun Schritt für Schritt die fehlenden Funktionen implementieren.

Programmiere nun analog zur Job-Page die Company-Page, also `routes/companies/+page.svelte` und `routes/companies/+page.server.js`:



Tipps zum Vorgehen:

- Für diese Aufgabe kannst du einen Grossteil des Codes aus `routes/jobs/+page.svelte` und `routes/jobs/+page.server.js` kopieren und anpassen.
- Nutze die Dokumentation auf <https://svelte.dev/>, um den vorhandenen Code zu verstehen, oder lasse ihn dir bei Bedarf von Copilot, Codex, ChatGPT etc. erklären.
- Nutze das **Cheatsheet** aus dem Modul *Prototyping*. Dort findest du Svelte-Beispielcode, den du für dieses Projekt verwenden und anpassen kannst.
- Du kannst bei Bedarf das Frontend wie gewohnt mit den Developer Tools im Browser debuggen. (Getestet mit Chrome, bei anderen Browsern können Variablen eventuell nicht überwacht werden.) Die dafür benötigte Konfiguration ist bereits in der Datei `vite.config.ts` vorhanden.

j) Abgabe

- a. Pushe dein Spring Boot Projekt (inkl. Frontend) in das Github Repository, welches du schon für die vorherigen Übungen verwendet hast.
- b. Gib den Link zur veröffentlichten API-Dokumentation auf Moodle ab (<https://documenter.getpostman.com/view/...>)